

ESTUDO DIRIGIDO

MLP Backpropagation com Dry Bean Dataset

Explicação Temática Completa

Disciplina: Inteligência Computacional

Autores: Yago e Davi

2026

Documento de referência teórica e prática sobre Redes Neurais Artificiais MLP treinadas com o algoritmo de Backpropagation, aplicadas ao problema de classificação multiclasse do Dry Bean Dataset (UCI Machine Learning Repository).

1. Introdução ao Aprendizado de Máquina Supervisionado

O Aprendizado de Máquina (Machine Learning) é um ramo da Inteligência Artificial que desenvolve algoritmos capazes de aprender padrões a partir de dados, sem serem explicitamente programados para cada tarefa. Dentro dessa área, o **Aprendizado Supervisionado** é a abordagem mais utilizada em problemas práticos: o modelo recebe exemplos rotulados (pares entrada-saída) e aprende a generalizar para novas entradas não vistas durante o treinamento.

Principais paradigmas supervisionados:

- **Classificação:** Mapear uma entrada a uma dentre N classes discretas. Ex: identificar variedades de feijão.
- **Regressão:** Mapear uma entrada a um valor contínuo. Ex: prever preço de um imóvel.

A base utilizada neste trabalho, o **Dry Bean Dataset**, é um exemplo clássico de classificação multiclasse: a partir de 16 atributos geométricos extraídos de imagens de grãos, o modelo deve classificar corretamente a variedade de feijão entre 7 categorias possíveis.

2. Redes Neurais Artificiais – Perceptron Multicamadas (MLP)

2.1 Histórico e Motivação Biológica

O Perceptron Multicamadas (MLP – *Multilayer Perceptron*) é inspirado na estrutura do neurônio biológico: unidades de processamento (neurônios artificiais) organizadas em camadas, onde cada neurônio recebe sinais de entrada, aplica uma transformação não-linear (função de ativação) e propaga o resultado para a camada seguinte. Isso permite que a rede aprenda representações hierárquicas dos dados.

2.2 Arquitetura da Rede

Uma MLP é composta por três tipos de camadas:

Camada	Descrição	Neste Trabalho
Entrada	Recebe os atributos normalizados da entrada.	16 neurônios (16 features)
Oculto(s)	Aprende representações intermediárias dos dados	1–3 camadas, 10–128 neurônios
Saída	Produz a distribuição de probabilidade sobre as classes	7 neurônios (7 classes de feijão)

2.3 Funções de Ativação

As funções de ativação introduzem não-linearidade à rede, permitindo que ela aprenda padrões complexos. Sem elas, a composição de camadas lineares seria equivalente a uma única transformação linear.

Função	Fórmula	Intervalo	Uso Típico
ReLU	$\max(0, x)$	$[0, +\infty)$	Camadas ocultas (padrão moderno)
Sigmoid	$1 / (1 + e^{(-x)})$	$(0, 1)$	Classificação binária (saída)
Tanh	$(e^x - e^{(-x)}) / (e^x + e^{(-x)})$	$(-1, 1)$	Camadas ocultas (alternativa)
Softmax	$e^{(x_i)} / \sum(e^{(x_j)})$	$(0,1)$, soma=1	Classificação multiclasse (saída)

Neste trabalho, foi utilizada ReLU nas camadas ocultas por sua eficiência computacional e por evitar o problema de vanishing gradient. A camada de saída utiliza Softmax, implicitamente calculada pela função de perda categorical cross-entropy do scikit-learn.

3. Algoritmo de Backpropagation

3.1 Conceito e Motivação

O **Backpropagation** (Retropropagação do Erro) é o algoritmo de treinamento central das MLPs. Seu objetivo é ajustar os pesos da rede de modo a minimizar a função de perda, calculada como a discrepância entre as saídas previstas e os rótulos reais. O algoritmo opera em duas fases:

- 1. Forward Pass (Propagação Direta):** Os dados de entrada percorrem a rede camada a camada, gerando as previsões.
- 2. Backward Pass (Retropropagação):** O erro é propagado de volta pela rede usando a Regra da Cadeia do Cálculo Diferencial, calculando o gradiente da perda em relação a cada peso.
- 3. Atualização dos Pesos:** Os pesos são atualizados na direção oposta ao gradiente, reduzindo a perda iterativamente ao longo das épocas.

3.2 Regra de Atualização dos Pesos

Fórmula geral (Gradiente Descendente):

$$w(t+1) = w(t) - lr \times (dL / dw)$$

Onde: **w(t)** é o peso atual, **lr** é a taxa de aprendizado, e **dL/dw** é o gradiente da função de perda L em relação ao peso w.

3.3 Taxa de Aprendizado e Momentum

A **taxa de aprendizado (lr)** controla o tamanho do passo de atualização dos pesos. Valores muito altos causam instabilidade e divergência; valores muito baixos resultam em convergência lenta. O **termo momento (momentum)** acelera a convergência ao acumular gradientes de iterações anteriores:

$$v(t+1) = \text{beta} * v(t) + lr * (dL / dw)$$

$$w(t+1) = w(t) - v(t+1)$$

*Neste experimento, o Grid Search identificou **lr = 0.1** e **momentum = 0.9** como a melhor combinação, convergindo em apenas ~28 épocas — enquanto **lr = 0.001** com **momentum = 0.5** necessitou de 200 épocas sem convergir totalmente.*

3.4 Otimizador Adam

O **Adam** (Adaptive Moment Estimation) é uma variante moderna do gradiente descendente que adapta automaticamente a taxa de aprendizado para cada parâmetro, combinando momentum de primeira e segunda ordem. Foi utilizado na Questão 1 com **lr = 0.001**, demonstrando boa estabilidade mesmo com diferentes inicializações de pesos (desvio

padrão de acurácia < 0.001).

4. Funções de Perda

A função de perda quantifica o erro entre as previsões da rede e os rótulos verdadeiros, guiando o processo de otimização. Para classificação multiclasse, a mais adequada é:

4.1 Entropia Cruzada Categórica

$$L = -1/N * \sum_i \sum_c y_{ic} * \log(p_{ic})$$

Onde N é o número de exemplos, C o número de classes, y_{ic} é 1 se o exemplo i pertence à classe c (e 0 caso contrário), e p_{ic} é a probabilidade prevista. Esta função penaliza mais fortemente previsões incorretas com alta confiança.

5. Estratégias de Validação e Métricas

5.1 Divisão Treino / Validação / Teste

Uma boa prática em ML é reservar parte dos dados exclusivamente para avaliação final. Neste trabalho, foi adotada a divisão estratificada:

Conjunto	Proporção	Uso
Treino	60%	Ajuste dos pesos da rede (Q1 a Q5)
Validação	20%	Seleção de hiperparâmetros e topologia
Teste	20%	Avaliação final – usado apenas em Q6 e Q7

5.2 Validação Cruzada K-Fold

A **Validação Cruzada K-Fold** divide o dataset em K subconjuntos iguais (folds). A cada iteração, um fold é usado como teste e os demais como treino. O processo repete-se K vezes, garantindo que todos os exemplos sejam usados para teste uma vez. A média e o desvio padrão das métricas oferecem uma estimativa estatisticamente robusta do desempenho real do modelo.

Neste trabalho: $k = 10$ folds estratificados, garantindo proporcionalidade das classes em cada fold. Resultado: F1-Macro médio = 0.9210 ± 0.0023 , demonstrando alta consistência.

5.3 Métricas de Avaliação

Métrica	Fórmula	Interpretação
---------	---------	---------------

Acurácia	Acertos / Total	Proporção geral de classificações corretas
Precisão	$VP / (VP + FP)$	Qualidade das predições positivas
Revocação	$VP / (VP + FN)$	Cobertura dos positivos reais
F1-Macro	$2 * (P * R) / (P + R)$, média por classe	Equilíbrio entre precisão e revocação
Cross-Entropy	$-\sum(y_real * \log(y_pred))$	Divergência entre dist. real e prevista

6. O Dry Bean Dataset

O **Dry Bean Dataset** é um dataset público disponibilizado no UCI Machine Learning Repository. Contém imagens de grãos de feijão seco de 7 variedades diferentes, das quais foram extraídos atributos numéricos que descrevem as propriedades geométricas e morfológicas de cada grão.

Característica	Detalhe
Nome da base	Dry Bean Dataset (UCI ML Repository – ID 602)
Tipo do problema	Classificação multiclasse supervisionada
Número de exemplos	~13.611 instâncias
Número de atributos	16 atributos numéricos contínuos
Número de classes	7 variedades de feijão seco
Classes	Seker, Barbunya, Bombay, Cali, Dermason, Horoz, Sira
Pré-proc. recomendado	Normalização Z-score ou Min-Max; divisão estratificada

Principais Atributos de Entrada:

- **Area:** Número de pixels da região do grão na imagem.
- **Perimeter:** Comprimento da fronteira do grão.
- **MajorAxisLength / MinorAxisLength:** Comprimentos dos eixos da elipse que melhor se ajusta ao grão.
- **Eccentricity:** Excentricidade da elipse equivalente (0 = círculo perfeito, 1 = linha reta).
- **AspectRatio:** Razão entre os eixos maior e menor.
- **Compactness:** Medida de quão próximo o grão é de um círculo.
- **ShapeFactor1–4:** Fatores de forma adimensionais derivados de combinações dos atributos acima.

A complexidade intermediária do dataset — com sobreposição parcial entre classes como Dermason e Sira — torna-o ideal para explorar aspectos de overfitting, underfitting, seleção de atributos e regularização em MLPs.

7. Tópicos Experimentais Abordados

7.X – Inicialização de Pesos (Q1)

A inicialização aleatória dos pesos influencia a trajetória de treinamento. 5 seeds diferentes foram testadas, revelando baixa sensibilidade da arquitetura (128, 64) à inicialização com otimizador Adam.

7.X – Hiperparâmetros de Aprendizado (Q2)

O Grid Search sobre LR e Momentum mostrou que LR = 0.1 com Momentum = 0.9 oferece o melhor equilíbrio entre velocidade (28 épocas) e estabilidade. LR = 0.5 causou divergência.

7.X – Topologia da Rede (Q3)

Redes com 2 camadas e 80 neurônios obtiveram o melhor F1-Macro (0.923) no conjunto de validação. O aumento excessivo de neurônios não gerou ganhos significativos, indicando risco de overfitting.

7.X – Quantidade de Dados (Q4)

Com apenas 20% dos dados o modelo já atinge F1 = 0.868. A curva de generalização satura em torno de 80%, sugerindo que dados adicionais além desse ponto trazem ganhos marginais.

7.X – Seleção de Atributos (Q5)

AspectRatio e Compactness apresentaram maior importância por permutação. O subconjunto completo (16 atributos) obteve o melhor F1, com a remoção de 5 atributos aleatórios degradando significativamente o desempenho.

7.X – Validação Cruzada K-Fold (Q7)

A validação 10-fold confirmou a robustez do modelo: F1-Macro = 0.9210 ± 0.0023 , demonstrando que o desempenho observado é consistente e não dependente de uma divisão específica.

8. Conclusão

Este trabalho demonstrou, de forma sistemática, que a **rede neural MLP treinada com Backpropagation** é uma ferramenta poderosa para problemas de classificação multiclasse. A configuração final (topologia (30,), LR = 0.1, Momentum = 0.9, 16 atributos) alcançou **F1-Macro médio de 92,10%** na validação cruzada, com baixa variância entre folds.

Principais Conclusões:

- ✓ ReLU + Softmax são escolhas sólidas para classificação multiclasse com MLPs.
- ✓ O otimizador Adam mostrou-se estável para exploração inicial; SGD com momentum é eficiente para ajuste fino.
- ✓ A arquitetura de 1 camada com 30 neurônios foi surpreendentemente competitiva com redes maiores.
- ✓ A normalização Z-score foi essencial para a convergência adequada do modelo.
- ✓ O dataset Dry Bean, com sua complexidade intermediária, é ideal para estudos de regularização e generalização.